

问题 1: SGLang 的 RadixAttention/RadixCache 解决什么问题?

解决前缀复用问题。传统方式下,多个请求如果 prompt 开头相同,需要重复计算。RadixAttention 用基数树 (Radix Tree) 管理 KV Cache,自动识别和复用相同前缀的 KV,减少重复计算,降低 TTFT,提高吞吐量。

以上是 AI 给的定义,我认为它的作用就是把已经算过的东西自动存起来,就好像是我打算法时用的哈希表,这能够降低时间复杂度,从而更快地回答,进而同时服务更多人。

问题 2: SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分?

page-sized KV cache 对应 pipeline 里的 KV Cache 管理部分 (把 KV 存储按页划分,方便内存管理)。

prefix reuse 对应 pipeline 里的 RadixCache / prefill 部分 (复用前缀,减少重复 prefill 计算)。

要我说,前者就是把一大坨 KV 切成固定的几块,每块叫一个 page

后者就是找出哪几块的前缀是相同的,把重复的省掉,提高效率

它们两个就是配合打组合技来的,page-sized KV cache 把一坨 KV 掰开揉碎给 prefix reuse 吃,prefix reuse 看到有重复的就不吃,减少不必要的消化

问题 3: vLLM 的 PagedAttention 解决什么问题? 和 SGLang 的 RadixAttention/pagedKV 有什么联系与差异?

PagedAttention 解决的是 KV Cache 内存碎片和浪费问题,把 KV Cache 分成固定大小的“页”,像操作系统管理内存一样管理,提高显存利用率。

联系: SGLang 的 page-sized KV cache 借鉴了 vLLM PagedAttention 的分页思想,都为了高效管理 KV 内存。

差异: vLLM 侧重分页内存管理 (解决碎片); SGLang 的 RadixAttention 侧重前缀共享和复用 (用基数树自动匹配相同前缀,减少重复计算)。两者可以结合使用。我觉得就是 SGLang 将 vLLM PagedAttention 进化实现更高级的前缀复用